

The Complete RemiMinder User Journey & Tech Stack

Part 1: User Login (Patient or Caregiver)

This section details how users access the system using existing Phase 1 infrastructure.

1.1 What the User Sees (Flutter App)

The user is presented with a clear choice on the mobile interface.

- **User Action:** Taps either **"Patient Login"** or **"Caregiver Login"** and enters credentials.

1.2 Behind the Scenes (Authentication)

Step	Location	Action	Technology
1. Submission	Flutter App	User submits form $\rightarrow$ Calls POST /auth/login (FastAPI)	Flutter (New Frontend)
2. Validation	FastAPI Backend	Uses Supabase Auth (existing code) to validate credentials.	Supabase Auth (Reuse)
3. Token Return	FastAPI	Returns a JWT token .	FastAPI (Reuse)
4. Routing	Flutter App	Saves token locally (SharedPreferences); checks user role; routes to PatientHome or CaregiverHome .	Flutter

- **Note:** No **Cloud SQL** or **AI Agents** are needed at this stage.

Part 2: Patient Home Screen

2.1 What the Patient Sees (Flutter App)

The dashboard provides immediate access to core functions and summaries.

2.2 Behind the Scenes (Data Fetching) ⚙️

Step	Location	Action	Technology
1. Screen Load	Flutter App	Calls existing Phase 1 APIs: GET <code>/api/reminders?user_id=123</code> and GET <code>/api/visit-summaries</code> .	Phase 1 APIs (Reuse)
2. Data Query	FastAPI Backend	Queries the Phase 1 Supabase database.	Supabase (Reuse)
3. Display	Flutter App	Displays the list of reminders and visit history.	Flutter UI (New)

Part 3: Recording a Visit (The Full AI Pipeline)

This is the core feature utilizing the new **Multi-Agent System** and **Offline-First** architecture.

3.1 Step-by-Step Breakdown 🎙️

Step	Location & Code	Action & Detail	Technology Stack
1. Audio Capture	Flutter App (<code>audio_recorder_service.dart</code>)	Records conversation; saves audio file locally (e.g., <code>/visit_456.mp3</code>).	Flutter audio recording package
2. Upload Audio	Flutter $\rightarrow$ FastAPI (<code>server.py</code>)	Sends audio file via POST <code>/upload-audio</code>	FastAPI (Existing Route)

		as <code>multipart/form-data</code> .	
3. Transcription	FastAPI Backend (<code>server.py</code>)	Calls OpenAI Whisper API to convert speech to text (transcript).	OpenAI Whisper (Reuse)
4. Save Transcript	FastAPI Backend (<code>db_service.py</code>)	Saves transcript text and creates the visit record in the database.	Supabase / Cloud SQL (Migration target)
5. AI Summarization	FastAPI Backend (<code>ai_service.py</code>)	Triggers the Multi-Agent System.	Vertex AI, Gemini 2.5 Flash, LangChain (NEW)
6. Save Summary	FastAPI Backend (<code>db_service.py</code>)	Saves the extracted summary and reminders into the database tables (<code>visit_summaries</code> , <code>reminders</code>).	Supabase / Cloud SQL
7. Sync to Flutter	Flutter App	PowerSync detects new data in Cloud SQL, downloads the summary and reminders, and updates the local SQLite DB.	PowerSync (NEW)

3.2 Agent #1: Visit Processor Agent

Component	Detail

Location	services/agents/visit_processor_agent.py
Input	Raw transcript text.
What it Does	Calls Gemini 2.5 Flash (via Vertex AI) using prompt engineering to extract structured data: Medications , Diagnoses , Action Items , and detailed Reminder Objects (including time/recurrence).
Output	Structured JSON containing the summary and detailed reminder objects.

3.3 Visual Summary of Recording Flow

This flowchart illustrates the data lifecycle for a recorded visit.

Part 4: RemiScan Feature

This feature handles paper prescriptions, introducing **OCR** and additional agents.

4.1 Step-by-Step Breakdown

Step	Location & Code	Action & Detail	Technology Stack
1. Photo Capture	Flutter App (remiscan_screen.dart)	Opens camera, takes photo of prescription, saves locally.	Flutter camera package
2. Store Image	FastAPI $\rightarrow$ Cloud Storage (route/remiscan.py)	Uploads photo to Google Cloud Storage bucket; saves URL to DB.	Google Cloud Storage (NEW)
3. OCR / Parsing	Agent #2: RemiScan Agent	Calls Google Vision API to get text from the photo, then uses Gemini 2.5 Flash to parse the text	Google Vision API (NEW)

		into structured medication details.	
4. Conflict Check	Agent #3: Conflict Detector	Queries DB for patient's current meds; calls Gemini to check for drug interactions and safety warnings.	Agent Orchestration (NEW)
5. Create Reminders	Agent #4: Reminder Generator	Uses parsed data to create structured reminder objects (e.g., time: 20:00, recurrence: daily).	Agent Orchestration
6. Sync to Flutter	FastAPI $\rightarrow$ PowerSync	New medications and reminders are saved to the database and synced to the Flutter app via PowerSync .	PowerSync

4.2 Visual Summary of RemiScan Flow

This flowchart illustrates the visual data processing pipeline.

Part 5: Reminders & Notifications

5.1 Patient Experience

The patient receives a push notification ("Time to take Lisinopril 10mg") at the scheduled time, opens the app, and marks the reminder complete.

5.2 Notification System (Supporting Services)

Component	Purpose	Flow / Trigger	Technology Stack
Cloud Scheduler	Triggers reminder checks.	Runs every minute $\rightarrow$ calls FASTAPI <code>/api/cron/trigger-reminders</code> .	Cloud Scheduler (NEW)

Reminder Scheduler	Checks due reminders and sends alerts.	Queries DB for reminders due now; sends appropriate messages.	AWS SES (Email - Reuse), Firestore Cloud Messaging (FCM) (Push - NEW)
FCM	Mobile push delivery.	Backend sends notification payload to FCM \$rightarrow\$ FCM delivers to Flutter app.	Firestore Cloud Messaging (NEW)

Part 6: Caregiver Experience

6.1 Caregiver Interface

The CaregiverHome screen provides a summarized view of the patient's activity, upcoming reminders, and critical alerts (e.g., "⚠ Missed 2 medications").

6.2 Alert Generation (Agent #5)

Agent	Purpose	Trigger	Data Access
Agent #5: Follow-up Automation	Generates critical alerts and triggers follow-ups.	Patient skips medication 2+ times; detects prescription renewal needed.	Fetches patient history via established Caregiver-Patient linkage (Phase 1).
Action	Sends email (AWS SES) and push notification (FCM) to the caregiver.		

Complete Technology Stack Summary

All Agents and Runtime Environment

Agent Name	Code Location	Runtime Environment	Core Technologies

Visit Processor	services/agents/...	FastAPI (Cloud Run)	Vertex AI, Gemini 2.5 Flash, LangChain
RemiScan Agent	services/agents/...	FastAPI (Cloud Run)	Vertex AI, Gemini 2.5 Flash, Google Vision API
Conflict Detector	services/agents/...	FastAPI (Cloud Run)	Vertex AI, Gemini 2.5 Flash
Reminder Generator	services/agents/...	FastAPI (Cloud Run)	Vertex AI, Gemini 2.5 Flash
Follow-up Automation	services/agents/...	FastAPI (Cloud Run)	Vertex AI, Gemini 2.5 Flash

Supporting Services Map

Service	Purpose	When Used	Phase
Supabase Auth	User login/JWT	Login	Phase 1 (Reuse)
Cloud Storage	Store audio/photos	After mobile upload	Phase 2 (New)
Cloud SQL	Main database (HIPAA-compliant)	All data operations	Phase 2 (New)
Vertex AI	Host and run AI agents	Process visits/scans	Phase 2 (New)

Firestore (FCM)	Push notifications	Reminders/Alerts	Phase 2 (New)
AWS SES	Email notifications	Reminders/Alerts	Phase 1 (Reuse)
Cloud Scheduler	Trigger reminder checks	Every minute	Phase 2 (New)
Stripe	Payments/Subscriptions	Premium signup	Phase 2 (New)
PowerSync	Offline sync (Cloud SQL $\rightarrow$ Flutter)	Always	Phase 2 (New)

Phase 1 vs Phase 2 Components

Phase 1 (Reuse - No Changes)	Phase 2 (New - To Build)
✓ Supabase Auth (login/signup)	 Flutter mobile app
✓ FastAPI structure (main.py, routes/)	 PowerSync (offline sync)
✓ AWS SES (email notifications)	 Cloud SQL (migrate from Supabase)
✓ Existing APIs (/api/reminders , etc.)	 Multi-agent system (all 5 agents)
✓ OpenAI Whisper	 Vertex AI integration
	 Cloud Storage (audio/photo storage)

	 Firestore (FCM) (push notifications)
	 RemiScan feature (OCR/Vision API)